Adaptive Phase-Switching for Communication-Efficient Federated LoRA Fine-Tuning

Jerry Adams Franklin

***Abstract*—Federated fine-tuning of large language models with low-rank adaptation reduces per-client trainable parameters, but client-to-server communication remains the dominant cost. Existing accounting for federated LoRA protocols omits the asymmetric transition round when a protocol changes aggregation mode, and reports savings that ignore grouped-query attention shapes. This paper measures per-round upload and download bytes for a bidirectional B-only federated LoRA protocol and places five published methods on a single communication-quality frontier. The frontier has a knee, which ReverseAdaptive locates by monitoring the relative improvement in global training loss against a dimensionless threshold rather than by fixing a phase boundary in advance. On TinyLlama-1.1B-Chat with Alpaca, ReverseAdaptive attains 40.5 percent measured round-trip savings over FLoRA at a held-out instruction-following loss cost of 0.0063. It outperforms FFA-LoRA, which freezes the first of the two LoRA factors at initialization, by 0.0182 in held-out loss, more than twenty times the largest per-method seed standard deviation on that metric, so learning that factor before freezing it produces better adapters. The same threshold transfers across model scales without retuning, and the quality cost of the transition is stable across the two datasets tested.**

***Index Terms*—Federated learning, low-rank adaptation, communication efficiency, parameter-efficient fine-tuning, large language models.**

I. Introduction

Federated learning enables model training across distributed clients without sharing raw data, making it attractive for privacy-sensitive applications of LLMs [1]. Fine-tuning with LoRA [2] substantially reduces the number of trainable parameters per client, but federated LoRA aggregation still transmits nontrivial bytes per round. With 10 clients participating in every round across 15 rounds, the cumulative cost determines whether a protocol is deployable in bandwidth-constrained settings.

Existing federated LoRA methods differ in what they aggregate and when. FedIT [3] applies FedAvg [4] directly to LoRA's A and B matrices, paying full bidirectional cost per round. FFA-LoRA [5] freezes the A matrix server-side and aggregates only B, achieving a theoretical 50% upload savings once A is frozen. FLoRA [6] stacks client LoRA modules and compresses via SVD, preserving product-space semantics at full bidirectional cost. FlexLoRA [7] supports heterogeneous client ranks and is orthogonal to the communication protocol studied here. These four report communication savings as parameter-count ratios rather than measured bytes; more recent work does measure transmitted volume directly [8], [9]. What remains uncharacterized is the asymmetric round that occurs when a protocol transitions between aggregation modes, and the use of an adaptive signal to select when that transition should happen.

This paper addresses these gaps with three contributions.

Bidirectional B-only protocol with measured byte tracking. The protocol implements both upload and download B-only transmission for Two-Phase and ReverseAdaptive aggregators. Per-round upload and download megabytes are recorded from the tensors actually transmitted. The transition round, in which the server seeds its frozen A from client uploads, is accounted for explicitly, producing a one-round asymmetry between when download savings begin and when upload savings begin.

ReverseAdaptive: adaptive phase-switching aggregator. The aggregator starts in FLoRA mode and monitors the relative per-round improvement in global training loss. When that relative improvement falls below a dimensionless threshold τ for the first time after a warmup period, the aggregator switches to FFA-LoRA permanently. Varying τ traverses the same communication-quality frontier as the fixed-K Two-Phase family without requiring K to be selected in advance.

A five-method frontier and its knee. Five published protocols are placed on one measured frontier at two model scales and on two datasets, evaluated by held-out instruction-following loss rather than by zero-shot benchmarks, which do not discriminate between protocols at the smaller scale. The frontier has a knee at ReverseAdaptive: savings beyond that point cost roughly five times more quality per point.

Headline numbers: Two-Phase $K{=}8$ saves 27.7% (TinyLlama-1.1B) and 26.0% (LLaMA-3.2-3B) over FLoRA. ReverseAdaptive saves 40.5% and 30.0 ± 4.0% respectively, and FFA-LoRA saves 61.9% at TinyLlama scale for 0.0182 more held-out loss than ReverseAdaptive. At LLaMA-3.2-3B, ReverseAdaptive and Two-Phase $K{=}8$ differ in final loss by 0.000012, roughly 350 times less than at 1.1B.

Code, configuration files, and scripts to reproduce every figure and table are publicly released; see the supplemental material.

II. Related Work

A. LoRA and Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) methods reduce the cost of adapting large pretrained models by training only

J. A. Franklin is an Independent Researcher, Rochester, NY 14450 USA (e-mail: jerry.adamsf@gmail.com). ORCID: 0009-0006-8470-8349.

a small subset of parameters. Adapter tuning [10] inserts small trainable modules between transformer layers. LoRA [2] decomposes weight updates into low-rank products BA, where B is $d \times r$ and A is $r \times k$ with r much smaller than $\min(d, k)$. This reduces trainable parameters substantially, enabling fine-tuning of large models without modifying base weights. QLoRA [11] further reduces memory by quantizing the frozen base model to 4-bit while training LoRA adapters in 16-bit. The resulting LoRA adapters are compact and composable, making them a natural fit for federated settings where both storage and communication are constrained.

B. Federated LoRA Aggregation

FedIT [3] applies standard FedAvg [4] to LoRA matrices: the global A and B are computed as weighted averages of client A and B matrices. The aggregation introduces a mismatch: the optimal global update is the product BA, not a product of separately averaged B and A. Averaging independently introduces aggregation error that grows with client heterogeneity.

FFA-LoRA [5] addresses the aggregation mismatch by freezing A at initialization and training only B across clients. Because A is constant, the server aggregates B matrices directly without product-space distortion. The theoretical upload saving is 50%. The original work also motivates the frozen-A design in terms of differential privacy. A critical difference from the protocol in this paper: FFA-LoRA freezes A at initialization, whereas the bidirectional B-only protocol allows A to be learned through a FLoRA phase before freezing, recovering expressivity at the cost of longer initial overhead.

FLoRA [6] preserves product-space semantics by stacking client LoRA modules horizontally and applying truncated SVD to recover a global low-rank adapter, reducing the aggregation bias present in FedIT. The tradeoff is communication cost: FLoRA transmits full $A{+}B$ payloads in both directions every round. FLoRA is the full-rank baseline in this paper's experiments.

FlexLoRA [7] extends federated LoRA to the heterogeneous-rank setting, where different clients use different LoRA ranks based on local compute budgets. FlexLoRA's contribution is orthogonal to this paper's; the bidirectional B-only protocol applies equally to homogeneous-rank clients.

C. Communication-Efficient Federated Learning

Communication efficiency in federated learning has been a central concern since FedAvg [4] introduced local multi-step updates to reduce communication rounds. The broader challenge of open problems in federated learning is surveyed by Kairouz et al. [1]. Subsequent work has pursued per-round payload reduction through gradient compression. QSGD [12] reduces the bit-width of transmitted gradients through stochastic quantization. Deep gradient compression [13] applies aggressive sparsification, sending only 0.1% of gradients per round with error feedback. FedPAQ [14] combines periodic averaging with quantization, showing that 8-bit transmission incurs negligible accuracy loss. These techniques are orthogonal to the B-only protocol studied here and could in principle be composed with it for additional savings.

D. Adaptive Aggregation and Scheduling

Several federated learning methods adapt their behavior based on observed training dynamics. FedProx [15] adds an adaptive proximal term to the local objective, controlling deviation from the global model. FedNova [16] normalizes local gradients by effective step count, adaptively correcting for heterogeneous local training. FedAdam and FedYogi [17] apply adaptive moment estimation at the server level, treating the aggregated pseudo-gradient as input to Adam-style updates. Curriculum scheduling in federated learning adjusts the aggregation frequency over training, structurally related to ReverseAdaptive's phase-switching in that both methods behave differently early vs. late in training. To the authors' knowledge, no prior federated LoRA work uses a loss-plateau signal to switch between distinct aggregation modes during training.

III. Method

A. Background and Notation

Each of N clients holds a local dataset. A shared base model has weight matrices W. LoRA [2] parameterizes updates as $\Delta W = BA$ where B is $d \times r$ and A is $r \times k$. In each federated round, clients train their local adapters for one epoch, upload some representation to the server, and receive a global state.

The communication cost of a round is the sum of upload bytes and download bytes. The B-only fraction of the full state is not the 50% a naive parameter count suggests, because grouped-query attention fixes the ratio of B to A parameters; Section III-D derives the exact fractions of 36% for TinyLlama-1.1B and 40% for LLaMA-3.2-3B.

B. Bidirectional B-Only Protocol

Phase 1 (FLoRA, rounds 1 to K). The server runs FLoRA aggregation [6]: client states are stacked and compressed via SVD to produce a global $(A_{\text{global}}, B_{\text{global}})$. Both upload and download payloads are full state.

Transition (round $K{+}1$). Clients still upload full state (A_i, B_i) because the server needs the A matrices to compute and cache A_{frozen}. The server runs FFA-LoRA [5] aggregation for the first time, caches A_{frozen}, and broadcasts the full global state.

Phase 2 steady state (rounds $K{+}2$ onward). Clients upload only B_i; the server reconstructs the full state as (A_{frozen}, B_i) before aggregation. The server broadcasts only B_{global}; clients retain A_{frozen} locally.

Look-ahead download accounting. This produces a one-round asymmetry: download savings begin one round before upload savings. The experiment runner records this consistently, ensuring reported totals are accurate rather than optimistic.

Algorithm 1 specifies the server-side decision logic per round.

Algorithm 1 Bidirectional B-only protocol (server, per round)

Require: round r, phase boundary K, frozen-A cache (initially empty)
Ensure: upload mode, broadcast mode

if $r \leq K$ **then** ▷ FLoRA phase
  upload ← full state (A, B)
  broadcast ← full state
else if $r = K + 1$ **then** ▷ transition: full upload seeds A_{frozen}
  upload ← full state (A, B)
  $A_{\text{frozen}} \leftarrow A$ from the first client's uploaded state
  broadcast ← B only ▷ clients retain A from round K
else ▷ FFA-LoRA steady state
  upload ← B only ▷ server prepends A_{frozen}
  broadcast ← B only ▷ clients retain A_{frozen} locally
end if

C. ReverseAdaptive Aggregator

ReverseAdaptive replaces the fixed boundary K with an adaptive loss-plateau signal. After a warmup period of W rounds (default W=5), it monitors the *relative* per-round loss improvement $\rho_r = (\ell_{r-1} - \ell_r)/\ell_{r-1}$. When $\rho_r < \tau$ for the first time, the aggregator switches to FFA-LoRA [5] mode permanently and instantaneously, with no intermediate phase. Because ρ_r is a ratio of losses, τ is a dimensionless fraction rather than a loss difference, and its interpretation does not depend on the absolute scale of the loss for a given model or dataset; Section V shows that this scale-free construction is what allows a single τ to transfer across model sizes without retuning. A complementary stability detector reverts to FLoRA if loss increases by more than 10% after switching; no reverts occurred across all 21 ReverseAdaptive runs in this study (3 IID seeds + 3 Non-IID α=0.5 + 5 Non-IID α=0.1 + 8 threshold-ablation + 3 LLaMA-3.2-3B IID + 1 no-switch sanity).

The choice of τ trades communication savings against final training loss. Small τ delays the switch; large τ triggers it at the first eligible round after warmup. Section IV-G characterizes this tradeoff empirically.

The loss-plateau signal is chosen for its simplicity, its availability without additional instrumentation, and the fact that it tracks the quantity a practitioner optimizes. Alternative signals such as the gradient norm of A or the singular value spectrum of stacked client matrices could provide more direct measures of whether A has converged, and are untested here.

Algorithm 2 specifies the aggregator's per-round decision logic.

Algorithm 2 ReverseAdaptive aggregator (per round)

Require: round r, current loss ℓ_r, prior loss ℓ_{r-1}, warmup W, threshold τ, current mode m, `client_states`
Ensure: `global_state`, updated mode m

$\rho_r \leftarrow (\ell_{r-1} - \ell_r) \,/\, \ell_{r-1}$ ▷ relative improvement; τ dimensionless
if $m =$ FLoRA **and** $r > W$ **and** $\rho_r < \tau$ **then**
  $m \leftarrow$ FFA-LoRA
  log event $(r,$ `switch_to_ffa`$)$
else if $m =$ FFA-LoRA **and** $\ell_r > 1.1 \cdot \ell_{r-1}$ **then** ▷ stability revert
  $m \leftarrow$ FLoRA
  reset A_{frozen}
  log event $(r,$ `revert_to_flora`$)$
end if
if $m =$ FFA-LoRA **then**
  return FFA-LoRA-AGGREGATE(`client_states`)
else
  return FLoRA-AGGREGATE(`client_states`)
end if

D. Implementation Details

Byte tracking. Per-round upload and download megabytes are recorded in `results.json`. Byte counts use the dtype in which parameters are transmitted: float32 (4 bytes/param) for TinyLlama-1.1B and float16 (2 bytes/param) for LLaMA-3.2-3B, whose LoRA parameters are trained in float32 and cast back to the base dtype for transmission. A full TinyLlama LoRA state is 2,252,800 parameters over 22 layers, or 9,011,200 bytes per client per direction. All models are optimized using AdamW [18].

Frozen-A provenance. At the transition round the server seeds A_{frozen} from the first client's uploaded state rather than from an average across clients. Because every client begins that round from the same broadcast global state, this is the aggregated global A of the preceding round plus one epoch of local adaptation at learning rate 10^{-4}. Client ordering is fixed for the duration of a run, so the choice is deterministic and reproducible. Two-Phase and ReverseAdaptive share this rule, since both dispatch to the same FFA-LoRA aggregator at their boundary. Averaging A across clients at the transition round is a plausible alternative that this work does not test.

GQA B-fraction. For TinyLlama-1.1B with rank r=16 targeting {`q_proj`, `v_proj`}, the projections adapted in the original LoRA formulation [2], accounting for GQA in the value projection, the B-only fraction of the full state is exactly 36% rather than the 50% a naive parameter count would suggest. This fraction is invariant to the size of the target set, since `q_proj` and `o_proj` share a shape and `k_proj` and `v_proj` share a shape under GQA; the ratio is fixed by the GQA configuration, not by how many projections are adapted. LLaMA-3.2-3B's different GQA configuration yields exactly 40%.

Partition mechanics. Alpaca and Dolly-15k carry no class labels, so non-IID partitions apply Dirichlet skew over a task-diversity proxy: each instruction s is assigned to one of ten buckets by character length, $\min(\lfloor |s|/50 \rfloor, 9)$, and a Dirichlet distribution with concentration $\alpha \in \{0.5, 0.1\}$ is applied over those buckets. This induces heterogeneity in task form rather than task semantics; Section VI notes that semantic heterogeneity is untested.

IV. Experiments and Results

TABLE I
EXPERIMENTAL SETUP.

Parameter	Value
Base models	TinyLlama-1.1B-Chat-v1.0 [19]; LLaMA-3.2-3B [20]
Datasets	Alpaca [21]; Dolly-15k [22]; 3000 samples each
Data partitions	IID; Dirichlet α=0.5; Dirichlet α=0.1
Clients / round	10 / 10 (full participation)
Rounds	15
Local epochs	1
Batch size	4 (TinyLlama), 2 (LLaMA-3.2-3B)
Grad. accumulation	4 (TinyLlama), 8 (LLaMA-3.2-3B); effective batch 16
Learning rate	10^{-4} (AdamW [18])
Max sequence length	256
LoRA rank / alpha	16 / 32
LoRA target modules	`q_proj`, `v_proj`
LoRA dropout	0.1
Hardware (primary)	Apple M4 Pro, 48 GB unified memory, MPS
Hardware (reproduction)	NVIDIA RTX 4090 (TinyLlama); A100 40 GB (LLaMA-3.2-3B)
Training dtype	float32 (TinyLlama-1.1B); float16 base + float32 LoRA (LLaMA-3.2-3B)
Transmitted dtype	float32 (TinyLlama-1.1B); float16 (LLaMA-3.2-3B)
Downstream eval	500 examples/benchmark, seed 42, log-likelihood
Held-out eval	500 examples, train indices 3001–3500, seq. len. 256
Benchmarks	MMLU [23], ARC-Easy [24], BoolQ [25], HellaSwag [26]
Seeds (TinyLlama IID)	{42, 123, 456}
Seeds (LLaMA-3.2-3B)	{42, 123, 456}
ReverseAdaptive defaults	`warmup_rounds`=5, τ=0.01, `stability_threshold`=1.1, `transition_rounds`=0 (instantaneous switch)

A. Setup

Table I summarizes the experimental configuration. The primary corpus of 34 runs was produced on a single Apple M4 Pro workstation, approximately 285 hours of compute, with no institutional cluster. Those runs were subsequently reproduced on commercially rented NVIDIA hardware, an RTX 4090 for TinyLlama-1.1B and an A100 40 GB for LLaMA-3.2-3B; the cross-backend comparison is reported in the supplemental material. The Dolly-15k replication and the FFA-LoRA and FedIT baseline runs were produced on the same rented hardware. Both TinyLlama-1.1B and LLaMA-3.2-3B experiments use three seeds for IID training.

Baselines. FFA-LoRA [5] and FedIT [3] are reimplemented within this codebase rather than obtained as released code from the original authors, and results for them should be read with that qualification. Following Sun et al., the FFA-LoRA implementation freezes A at initialization: client-side A parameters are set non-trainable before local optimization begins and are excluded from the optimizer parameter group, so clients train only B from the first round. Clients transmit B only, a measured 3,244,032 bytes per client per direction against 9,011,200 for full $A{+}B$ transmission. The former is $(32768{+}4096)\times 22$ parameters at 4 bytes each, exactly the B-only payload, and could not arise if A were being transmitted. FedIT applies FedAvg independently to A and B and transmits full state in both directions, so its communication volume matches FLoRA's by construction.

Prompt format. All examples, in training and in held-out evaluation, are rendered with a single template: `### Instruction:` followed by the instruction text, then `### Response:` followed by the output, truncated and padded to 256 tokens. Alpaca's optional `input` field, populated in roughly 40% of examples, is not included, and no separate template is applied to examples that carry it. Loss is computed over the full formatted sequence including prompt tokens, with padding masked to -100. This differs from the two-template formatting used in the original Alpaca release and shifts absolute loss values; because training and evaluation share the same code path, comparisons between methods are unaffected. Note that if the tokenizer aliases `pad_token` to `eos_token`, the end-of-sequence token is masked along with padding; this is consistent across all runs but is relevant to anyone recomputing the base-model reference values of Section IV-B.

Backend provenance. Communication totals are protocol-deterministic and identical across backends; loss values are not. Tables are therefore labelled with their corpus: the five-method frontier (Table II), the Dolly-15k replication (Table III), and all statistical tests use CUDA, while the threshold ablation (Table VI) and the LLaMA-3.2-3B results use MPS.

B. Held-Out Instruction-Following Metric

Final training loss measures fit to the federated objective, and the four zero-shot benchmarks do not discriminate between base and fine-tuned checkpoints at this scale (Section V-D). Quality is therefore also reported as held-out instruction-following loss. From each dataset we reserve the 500-example slice `train[3000:3500]`, disjoint from the 3000 examples partitioned across clients, and compute mean token-level cross-entropy under the prompt format described in Section IV-A. Each adapted checkpoint is scored against the unadapted base model, and we report $\Delta\ell_{\text{held}} = \ell_{\text{tuned}} - \ell_{\text{base}}$, so more negative values indicate better instruction following. Base-model reference values are 1.9352 (perplexity 6.9254) on Alpaca and 2.2608 (perplexity 9.5905) on Dolly-15k. Because the base values differ by dataset, held-out deltas are comparable within a dataset but not across datasets; the cross-dataset comparison in Section IV-D therefore compares between-method gaps rather than levels.

C. The Communication-Quality Frontier

Table II and Figure 1 place five published federated LoRA protocols on a single measured communication-quality frontier for TinyLlama-1.1B [19] under IID partitioning. The five methods occupy four distinct operating points. FLoRA [6] and FedIT [3] coincide at 2578.13 MB, since both transmit full A and B in both directions every round. Two-Phase K=8 reaches 1863.13 MB (27.73% savings), ReverseAdaptive at τ=0.01 reaches 1533.13 MB (40.53%), and FFA-LoRA [5] reaches 983.13 MB (61.87%). Across the four distinct points communication falls and both quality metrics degrade in the same order, so the frontier is monotone on each metric independently rather than only on the metric used to construct it.

Three features of Table II bear on the interpretation. The two quality metrics agree on the ordering of every pair they can resolve: across the four distinct operating points the ranking is identical on final training loss and on held-out instruction-following loss. The exception is the FLoRA and FedIT pair, which shares an operating point, where FedIT is nominally better on final loss and FLoRA nominally better on held-out loss; neither difference is statistically distinguishable (p=0.588

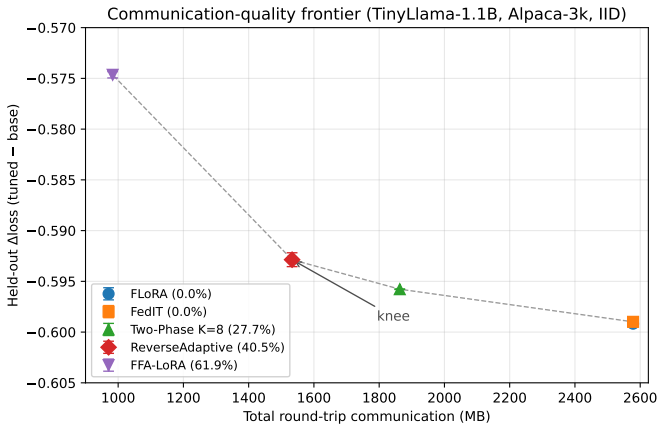

Fig. 1. Measured communication-quality frontier on Alpaca-3k, TinyLlama-1.1B, IID, CUDA corpus. Five published protocols at four distinct operating points; error bars are one sample standard deviation over three seeds, and communication is identical across seeds. Held-out Δloss is tuned minus base on the 500-example slice defined in Section IV-B; more negative is better. FLoRA and FedIT coincide at upper right. The segment from ReverseAdaptive to FFA-LoRA is markedly steeper than the segment preceding it, which Section V quantifies.

and p=0.425). The shape of the frontier is not an artifact of the metric chosen to construct it.

ReverseAdaptive outperforms FFA-LoRA by 0.0282 on final loss and 0.0182 on held-out loss, against per-method seed standard deviations of at most 0.0016 and 0.0008 respectively. Both protocols end training with A frozen and only B aggregated, and they differ only in whether A was learned first. Learning A through a FLoRA phase before freezing it produces materially better adapters than freezing A at initialization, at a cost of 550 MB in this configuration.

FedIT lands on FLoRA's communication volume exactly, to the byte, and within 0.0006 on both quality metrics. It contributes no new operating point, which is the intended result: FedIT is an independently implemented full-state protocol, so its coincidence with FLoRA is evidence that the byte accounting measures the protocol rather than an implementation artifact.

Loss differences between methods are small in absolute terms, 0.0141 between FLoRA and ReverseAdaptive at τ=0.01, but are resolved across three seeds because the per-seed spread is more than twenty times smaller. Paired t-tests on final loss give $p < 10^{-3}$ for every comparison against FLoRA except FedIT, which is correctly indistinguishable. With three seeds these tests have very low power, so effect sizes should be read alongside the p-values, and no multiplicity correction is applied in Table II; the supplemental material reports the full matrix on both metrics with the correction stated.

This frontier sweeps through the fixed-K Two-Phase operating points: on the MPS corpus, τ=0.002 lands on Two-Phase K=8 communication and τ=0.001 on K=10. Because total communication is a step function of the discrete switch round, at 110 MB per round, this correspondence is not backend-invariant; the supplemental material reports that both tight-τ settings fire one round earlier on CUDA. The robust claim is that the adaptive method traverses the same frontier as the fixed-K family without requiring K to be specified, rather than that it reproduces particular K values byte for byte.

D. Replication on Dolly-15k

Table III repeats the three-method comparison on Databricks Dolly-15k [22], an instruction dataset with a different length distribution and a broader task mix than Alpaca. Communication is identical to the Alpaca corpus, since the protocols are dataset-independent and ReverseAdaptive fired at round 6 on every seed of both datasets. The quantity of interest is therefore the quality cost of switching rather than the absolute loss.

Held-out deltas are not comparable across datasets, since the base model scores 1.9352 on the Alpaca slice and 2.2608 on the Dolly slice. Table III reports the gap to each dataset's own FLoRA baseline, which is comparable. ReverseAdaptive costs 0.0063 held-out loss on Alpaca and 0.0061 on Dolly, a shift of 0.0002, or 3.0% of the gap's own magnitude. The quality cost of the adaptive transition is stable across these two datasets.

Two-Phase K=8 costs 0.0034 on Alpaca and 0.0043 on Dolly, a shift of 25.5%. We do not read this as evidence that fixed-K switching transfers less well than adaptive switching. Three seeds on two datasets cannot resolve a difference in stability between two methods, and the claim supported here is limited to ReverseAdaptive's own cost being stable, not to a property of the protocol family.

A non-IID Dolly run at α=0.5 switched at round 6 on all three seeds and reached the same 1533.13 MB, matching the IID behaviour on both datasets. No Dolly FLoRA baseline was run under heterogeneity, so no quality cost is reported for that configuration.

E. Convergence and Switching Behavior

Figure 2 shows mean $\pm$ std loss trajectories across three seeds for FLoRA, Two-Phase K=8, and ReverseAdaptive. All three methods converge similarly through the FLoRA phase. Phase transitions are visible as slight slope changes but do not destabilize training. The final $\sim$0.01 loss gap accumulates gradually after the switch rather than appearing as a sharp discontinuity.

Figure 3 shows cumulative communication in megabytes against round number for TinyLlama-1.1B. FLoRA stays linear at 85.94 MB per round per direction throughout. Two-Phase K=8 maintains the same slope through round 9, then reduces as B-only transmission activates. ReverseAdaptive splits off at round 6 (at TinyLlama-1.1B scale), producing the steepest savings trajectory.

The no-switch sanity baseline (ReverseAdaptive with `switch_threshold` $= -1.0$, `warmup_rounds` $= 999$) produces final loss 1.2594342812 on seed 42 under the MPS corpus, matching the corresponding FLoRA run to 10 decimal places. The CUDA per-seed values in the supplemental material differ, as expected across backends; the guarantee is a within-backend statement. Upload and download match to the byte across all 15 rounds. Disabling the switch requires a negative threshold rather than a small positive one, for reasons set out in the supplemental material.

TABLE II

MEASURED COMMUNICATION-QUALITY FRONTIER. TINYLLAMA-1.1B, ALPACA-3K, IID, CUDA CORPUS, THREE SEEDS, MEAN ± ONE SAMPLE STANDARD DEVIATION. COMMUNICATION IS PROTOCOL-DETERMINISTIC AND IDENTICAL ACROSS SEEDS. HELD-OUT ΔLOSS IS TUNED MINUS BASE ON THE 500-EXAMPLE SLICE OF SECTION IV-B; MORE NEGATIVE IS BETTER. A REPORTED STANDARD DEVIATION OF 0.0000 INDICATES A SAMPLE STANDARD DEVIATION BELOW 10^{-4}. p-VALUES ARE PAIRED t-TESTS AGAINST FLORA ON FINAL LOSS; THE FULL MATRIX ON BOTH METRICS IS IN THE SUPPLEMENTAL MATERIAL. METHODS: FLORA [6], FEDIT [3], FFA-LORA [5]. THE MB COLUMN IS TOTAL ROUND-TRIP COMMUNICATION OVER 15 ROUNDS.

Method	MB	Savings	Final loss	Held-out Δ	p vs. FLoRA
FLoRA	2578.13	–	1.2608 ± 0.0005	-0.5992 ± 0.0001	–
FedIT	2578.13	0.00%	1.2602 ± 0.0016	-0.5990 ± 0.0002	0.588
Two-Phase K=8	1863.13	27.73%	1.2705 ± 0.0006	-0.5958 ± 0.0000	4.2×10^{-5}
ReverseAdaptive (τ=0.01)	1533.13	40.53%	1.2749 ± 0.0006	-0.5929 ± 0.0008	1.1×10^{-5}
FFA-LoRA	983.13	61.87%	1.3031 ± 0.0012	-0.5746 ± 0.0004	1.7×10^{-4}

TABLE III

DOLLY-15K REPLICATION. TINYLLAMA-1.1B, IID, CUDA CORPUS, THREE SEEDS, MEAN ± ONE SAMPLE STANDARD DEVIATION. GAP IS THE HELD-OUT ΔLOSS DIFFERENCE AGAINST THE SAME DATASET'S FLORA BASELINE, WHICH IS THE QUANTITY COMPARABLE ACROSS DATASETS; HELD-OUT LEVELS ARE NOT, SINCE BASE-MODEL LOSSES DIFFER (1.9352 ON ALPACA, 2.2608 ON DOLLY-15K). METHODS: FLORA [6], FEDIT [3], FFA-LORA [5]. THE MB COLUMN IS TOTAL ROUND-TRIP COMMUNICATION OVER 15 ROUNDS.

Method	MB	Final loss	Held-out Δ	Gap (Dolly)	Gap (Alpaca)
FLoRA	2578.13	1.6544 ± 0.0021	-0.5330 ± 0.0010	–	–
Two-Phase K=8	1863.13	1.6643 ± 0.0021	-0.5287 ± 0.0010	0.0043	0.0034
ReverseAdaptive (τ=0.01)	1533.13	1.6686 ± 0.0022	-0.5268 ± 0.0005	0.0061	0.0063

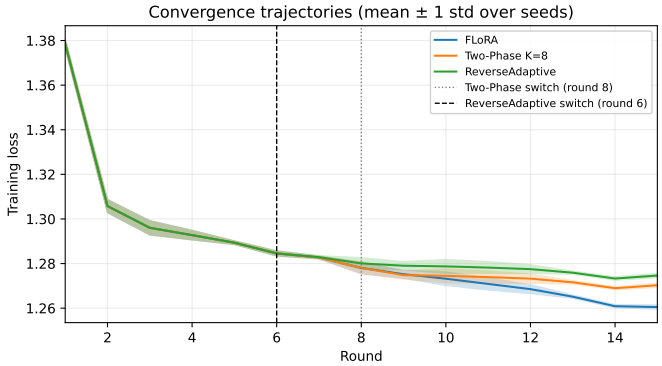

Fig. 2. Training loss trajectories on TinyLlama-1.1B IID. Mean ± 1 standard deviation over three seeds. Vertical dashed lines mark ReverseAdaptive's switch round (round 6) and Two-Phase K=8's phase boundary (round 8; the transition round, in which the upload is still full, is round 9). All three methods converge similarly through the FLoRA phase; the loss gap accumulates gradually after the switch.

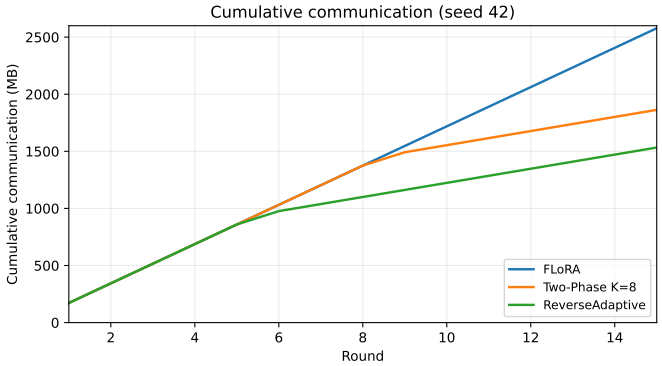

Fig. 3. Cumulative communication on TinyLlama-1.1B (seed 42, deterministic across seeds). Slope changes mark phase transitions: ReverseAdaptive at round 6, Two-Phase K=8 at round 9. The look-ahead download accounting causes the broadcast slope to change one round before the upload slope changes.

F. Downstream Evaluation

Tables IV–V and Figure 4 present downstream results on MMLU [23], ARC-Easy [24], BoolQ [25], and HellaSwag [26]. MMLU is excluded from TinyLlama-1.1B comparisons: base accuracy of 0.250 on 4-way multiple choice indicates chance-level performance, making the benchmark uninformative at 1.1B scale.

At TinyLlama-1.1B (Table IV), all federated methods cluster within 1.0 percentage points of each other across the three informative benchmarks. All methods regress modestly relative to the base model. The base model's BoolQ accuracy of 0.626 closely matches BoolQ's natural yes-class rate of approximately 0.62, suggesting the base exploits a yes-biased prior. Federated Alpaca-3k instruction tuning attenuates this prior, producing regression across all methods uniformly.

At LLaMA-3.2-3B scale, the three federated methods are statistically indistinguishable on downstream benchmarks. Mean accuracies across three seeds differ by at most 0.5 percentage points on any benchmark, and all cross-method differences lie within the seed-to-seed standard deviation of each individual method. The picture differs from the TinyLlama panel in two ways. The federated methods are flat to slightly positive relative to the base model at 3B rather than regressing as they did at 1.1B; the LLaMA base achieves MMLU 0.544, ARC-Easy 0.832, BoolQ 0.706, and HellaSwag 0.540, and federated methods either match or slightly exceed each of

TABLE IV
DOWNSTREAM ZERO-SHOT ACCURACY, TINYLLAMA-1.1B. MMLU OMITTED: BASE ACCURACY IS AT CHANCE FOR 4-WAY MULTIPLE CHOICE.

Method	ARC-Easy	BoolQ	HellaSwag	Δ vs. base (BoolQ)
Base (no adapter)	0.274	0.626	0.448	–
FLoRA	0.246	0.562	0.420	−6.4 pp
Two-Phase K=8	0.248	0.560	0.420	−6.6 pp
ReverseAdaptive	0.252	0.552	0.420	−7.4 pp
Spread (max–min)	0.006	0.010	0.000	–

TABLE V
DOWNSTREAM ZERO-SHOT ACCURACY, LLAMA-3.2-3B. MEAN ± 1 STANDARD DEVIATION ACROSS THREE RANDOM SEEDS (42, 123, 456). 500 EXAMPLES PER BENCHMARK. CROSS-METHOD SPREAD IS AT MOST 0.5 PERCENTAGE POINTS ON ANY BENCHMARK AND IS WITHIN THE SEED-TO-SEED STANDARD DEVIATION OF EACH METHOD.

Method	MMLU	ARC-Easy	BoolQ	HellaSwag
Base (no adapter)	0.544	0.832	0.706	0.540
FLoRA	0.558 ± 0.007	0.836 ± 0.007	0.736 ± 0.007	0.535 ± 0.003
Two-Phase K=8	0.558 ± 0.002	0.837 ± 0.008	0.738 ± 0.010	0.539 ± 0.004
ReverseAdaptive	0.557 ± 0.003	0.839 ± 0.008	0.739 ± 0.004	0.538 ± 0.004

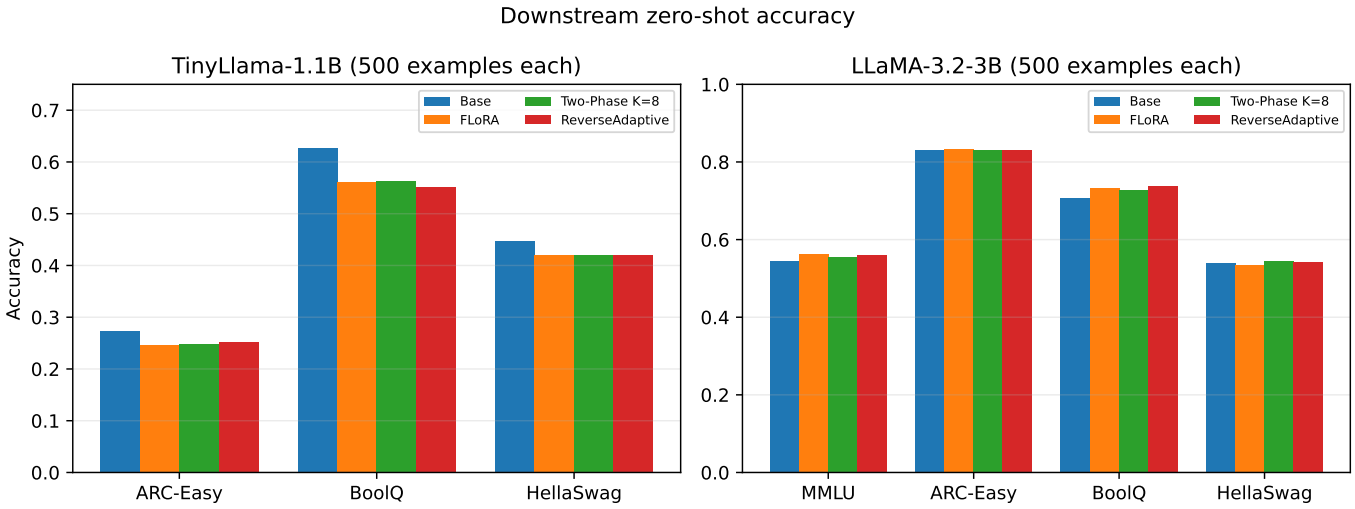

Fig. 4. Downstream zero-shot accuracy across two model scales (500 examples per benchmark). Left: TinyLlama-1.1B (seed 42 only; ARC-Easy, BoolQ, HellaSwag; MMLU omitted because base accuracy is at chance for 4-way multiple choice). Right: LLaMA-3.2-3B (mean across three seeds; MMLU, ARC-Easy, BoolQ, HellaSwag). Federated methods cluster within 1.0 percentage points at TinyLlama-1.1B, across the three informative benchmarks, and 0.5 percentage points at LLaMA-3.2-3B. TinyLlama accuracies are single-seed and from the MPS corpus; loss values elsewhere in this paper for the same configurations are three-seed and from the CUDA corpus.

these. The within-method seed variance is also smaller at 3B than the within-scale spread observed at TinyLlama, indicating that downstream benchmarks become more stable as model capacity grows.

G. Threshold Ablation

Table VI and Figure 5 characterize how τ controls the savings-quality tradeoff. Since τ bounds a relative rather than an absolute loss improvement, the values below are fractions of the previous round's loss: τ=0.01 requires the loss to fall by at least 1% in a round for the FLoRA phase to continue. The method has a sensitive regime for $\tau \leq 0.002$: at τ=0.002 the switch occurs at round 9, matching Two-Phase K=8; at τ=0.001 the switch occurs at round 11, matching Two-Phase K=10. For $\tau \geq 0.005$ the method saturates: the switch fires at round 6, the first round after warmup. All rows for $\tau \in \{0.005, 0.010, 0.020, 0.050, 0.100, 0.200\}$ produce identical results.

A practitioner seeking maximum savings can set τ anywhere in $[0.005, 0.2]$ without precision loss. The τ=0.001 and τ=0.002 operating points demonstrate that the adaptive method genuinely tracks the loss trajectory rather than saturating trivially.

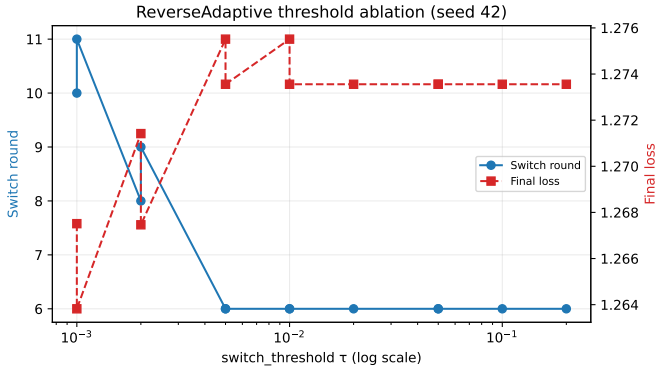

Fig. 5. ReverseAdaptive threshold sensitivity (TinyLlama IID, seed 42, MPS corpus). Communication is backend-invariant; loss values are not comparable with the CUDA results in Table II.

TABLE VI
REVERSEADAPTIVE THRESHOLD SENSITIVITY (TINYLLAMA IID, SEED 42).

τ	Switch round	Final loss	Total comm. (MB)	Savings
0.001	11	1.2638	2083.13	19.20%
0.002	9	1.2675	1863.13	27.73%
0.005	6	1.2736	1533.13	40.53%
0.010	6	1.2736	1533.13	40.53%
0.020	6	1.2736	1533.13	40.53%
0.050	6	1.2736	1533.13	40.53%
0.100	6	1.2736	1533.13	40.53%
0.200	6	1.2736	1533.13	40.53%

H. Scale Validation

Table VII and Figure 6 confirm that the Pareto structure observed at TinyLlama-1.1B is preserved at LLaMA-3.2-3B. Three random seeds (42, 123, 456) were run for each of FLoRA, Two-Phase K=8, and ReverseAdaptive, producing nine total runs. Final-round results (mean $\pm$ standard deviation): FLoRA reaches loss 1.3724 ± 0.0009 with 2625.0 MB total communication. Two-Phase K=8 reaches loss 1.3938 ± 0.0011 with 1942.5 MB (savings 26.0%). ReverseAdaptive reaches loss 1.3938 ± 0.0036 with 1837.5 ± 105.0 MB (savings 30.0 ± 4.0%). The protocol-deterministic communication for FLoRA and Two-Phase K=8 produces zero seed-to-seed variance; ReverseAdaptive's variance reflects across-seed differences in switch round (7, 8, 9 across the three seeds, mean 8.0 ± 1.0).

The communication savings are slightly lower at 3B than at 1.1B: 26.0% vs. 27.7% for Two-Phase K=8, and 30.0% vs. 40.5% for ReverseAdaptive. The B-only fraction of the full state is 40% at LLaMA-3.2-3B (vs. 36% for TinyLlama) due to different GQA configurations, meaning each B-only round saves a smaller fraction of the full payload at 3B scale.

At LLaMA-3.2-3B, ReverseAdaptive and Two-Phase K=8 reach final losses differing by 1.25×10^{-5}, roughly 350 times smaller than the 0.0043 separating them at TinyLlama-1.1B. A paired t-test over three seeds gives $p = 0.997$ with a 95% confidence interval of $[-0.0114, 0.0114]$. That interval contains the 1.1B effect size, so the test cannot distinguish no difference from a difference as large as the one observed at 1.1B, and the result is reported as a statement about effect size rather than as evidence of equivalence. ReverseAdaptive also saves 4.0 percentage points more mean communication than the hand-tuned configuration (30.0% against 26.0%), because it switches at round 7 on one of three seeds. The same threshold τ=0.01 used at TinyLlama-1.1B produced this behaviour with no scale-specific tuning.

TABLE VII
LLAMA-3.2-3B IID RESULTS. MEAN $\pm$ 1 STANDARD DEVIATION ACROSS THREE RANDOM SEEDS (42, 123, 456). REVERSEADAPTIVE SWITCH ROUNDS: 7, 8, 9. FLORA AND TWO-PHASE K=8 COMMUNICATION IS PROTOCOL-DETERMINISTIC; REVERSEADAPTIVE COMMUNICATION VARIES WITH SWITCH ROUND.

Method	Final loss	Total MB	Savings vs. FLoRA
FLoRA	1.3724 ± 0.0009	2625.0	0%
Two-Phase K=8	1.3938 ± 0.0011	1942.5	26.0%
ReverseAdaptive (τ=0.01)	1.3938 ± 0.0036	1837.5 ± 105.0	30.0 ± 4.0%

V. DISCUSSION

A. Implications for Federated LLM Deployment

In mobile or IoT federated settings where uplink bandwidth is scarce and metered, cutting round-trip communication by 30 to 40% for a held-out instruction-following loss cost of 0.0063 can change the economic viability of federated fine-tuning. Whether that trade is acceptable is a deployment decision rather than a universal one, and the frontier in Section IV-C is intended to let a practitioner make it explicitly: the same measurements show that accepting a further 21.3 points of savings costs roughly five times more quality per point. Kairouz et al. [1] identify communication as a fundamental open problem in federated learning at scale, and measured byte-level reporting is a prerequisite for addressing it.

B. Choosing an Operating Point

The Two-Phase protocol requires a practitioner to select K before training begins, without knowing when the loss plateau will occur for a given model-dataset combination. This is a hyperparameter that must be tuned per dataset, per model, and potentially per client population: precisely the kind of manual configuration that slows deployment in production federated systems. ReverseAdaptive replaces it by monitoring the signal that K was meant to approximate.

The criterion transfers across model scales because it is scale-free. Since τ bounds a relative loss improvement, the same numerical value encodes the same stopping condition at any absolute loss level, so it carries across models whose losses differ in magnitude; an absolute improvement threshold would not, since a fixed loss delta that plateaus one model's training would fire immediately or never on another. Using the same τ=0.01, ReverseAdaptive fires at round 6 on TinyLlama-1.1B and at a mean of round 8.0 ± 1.0 on LLaMA-3.2-3B, with no scale-specific retuning. Both sets of switch rounds match exactly across the MPS and CUDA backends (see the

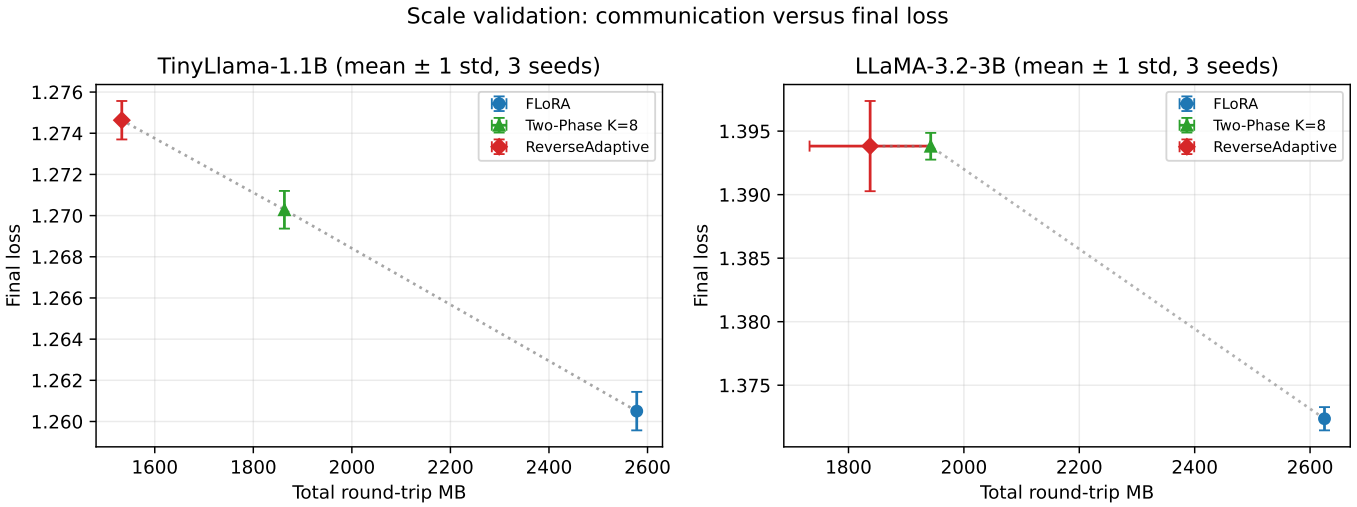

Fig. 6. Scale validation. Left: TinyLlama-1.1B (mean ± 1 standard deviation over three seeds). Right: LLaMA-3.2-3B (mean ± 1 standard deviation over three seeds). Both panels show the three federated methods on the same axes. Dotted polyline connects the three method points on each panel. The Pareto structure (downward slope from FLoRA to ReverseAdaptive) is preserved across scales. Y-axis ranges differ between panels because absolute training loss is model-dependent; the relative Pareto shape is the invariant being validated. ReverseAdaptive's larger error bar at LLaMA-3.2-3B reflects across-seed variance in switch round (7, 8, 9).

supplemental material), so the cross-scale comparison does not rest on corpora that disagree about when the switch occurred.

Two qualifications bound this result. At TinyLlama scale and τ=0.01 the switch fires at the first post-warmup round in every run, so adaptivity at this setting is documented in the threshold ablation (Section IV-G) rather than in the main TinyLlama runs, while the LLaMA-3.2-3B seeds give switch rounds 7, 8, and 9. Two model sizes and two datasets establish transfer without retuning, not a universal default for τ=0.01.

At 3B the adaptive and hand-tuned configurations reach final losses differing by 0.000012, roughly 350 times smaller than the 0.0043 that separates them at 1.1B. A paired t-test over three seeds gives $p = 0.997$. We report this as a statement about effect size rather than as evidence of equivalence: with three seeds the test has almost no power to detect a difference of the magnitude observed at 1.1B, so a large p-value cannot establish that none exists.

Where ReverseAdaptive sits on the frontier matters more than which fixed K it happens to match. Moving from FLoRA to ReverseAdaptive buys 40.5 percentage points of communication savings at a held-out cost of 0.0063, or 1.56×10^{-4} per point. Moving onward to FFA-LoRA buys a further 21.3 points at a cost of 0.0182, or 8.54×10^{-4} per point, 5.5 times more expensive per point saved. The frontier has a knee and ReverseAdaptive sits at it. A practitioner who takes the first tranche of savings and declines the second is making the trade the measured data supports, and the value of the adaptive rule is that it locates that point without K being guessed in advance.

C. Measured Bytes and Theoretical Parameter Counts

Where prior federated LoRA accounting reports parameter-count ratios or aggregate megabyte totals [3], [5], [6], [7], [8], [9], the contribution here is the isolation of two quantities that neither form of reporting separates.

Architectural choices such as GQA fix the B-only fraction in ways a naive parameter count does not anticipate. TinyLlama-1.1B's GQA makes B-only transmission save exactly 36% rather than the 50% a non-GQA count suggests, and LLaMA-3.2-3B's different GQA configuration yields exactly 40%. A theoretical 50% savings claim made for one model family does not transfer to another without architectural accounting.

The transition round carries a cost that parameter-ratio analyses omit. At the FLoRA-to-FFA-LoRA boundary the upload is full while the broadcast is already B-only, so exactly one directional payload is upgraded from B-only to full. On TinyLlama this costs 55.0 MB, and the figure does not depend on the switch round: totals that ignore the transition would be 928.1, 1478.1, and 1808.1 MB for switch rounds 1, 6, and 9, against measured totals of 983.1, 1533.1, and 1863.1. What varies with the number of remaining rounds is the penalty's share of the total, not its size. A protocol upgrading both directions at the boundary would pay 110.0 MB; the look-ahead broadcast halves it.

Total communication across the entire corpus is one closed-form expression in one discrete parameter. For a switching protocol with switch round s over N rounds, uploads are full for rounds $1 \ldots s$ and broadcasts for rounds $1 \ldots s-1$, so $2s-1$ directional payloads are full and the total is $55(2s-1) + 928.125$ MB on TinyLlama and $52.5(2s-1) + 1050$ MB on LLaMA-3.2-3B. FFA-LoRA is s=1, ReverseAdaptive s=6, Two-Phase K=8 is s=9, and Two-Phase K=10 is s=11; every measured total in this paper lands on that lattice to the byte. The $2s-1$ arises from the harvest rule rather than from the protocol: an implementation that froze the server's existing global A would upgrade no directional payload at the boundary and give $110(s-1) + 928.125$ instead. A non-

switching protocol has all $2N$ directional payloads full and is not a point on the lattice: FLoRA is the separate endpoint at $55 \times 30 + 928.125 = 2578.125$ MB. The lattice spacing of 110 MB per round is also why no continuous τ reproduces a fixed-K total robustly, as Section IV-C notes.

One artifact of this implementation deserves disclosure, because it charges the strongest baseline too much. FFA-LoRA freezes A at initialization, so every client's A equals the shared initial value at every round and never needs transmitting. The measured 983.1 MB nonetheless includes the 55.0 MB seeding upload, because the server harvests A from the first client's uploaded state rather than deriving it from the initialization (Section III-D). An implementation faithful to the design reaches the floor of 928.1 MB, raising FFA-LoRA's savings from 61.87% to 64.00%, widening the ReverseAdaptive-to-FFA-LoRA segment from 21.3 to 23.5 percentage points, and moving the marginal cost ratio from 5.5 to 5.0. The knee conclusion is unchanged. The same harvest rule governs ReverseAdaptive and Two-Phase, where the seeding upload is likewise avoidable in principle, since the server already holds the aggregated global A it broadcast in the preceding round; freezing that value instead is an untested alternative recorded in Section VI.

D. The Base-Model Regression Observation

Federated Alpaca-3k [21] instruction tuning regresses TinyLlama-1.1B [19] on all three informative zero-shot benchmarks: ARC-Easy by 2.2 to 2.8 percentage points depending on method, BoolQ by 6.4 to 7.4, and HellaSwag by 2.8 uniformly. At LLaMA-3.2-3B [20] the same procedure is flat-to-positive on all four benchmarks. The cross-scale contrast is consistent with a model-capacity explanation: TinyLlama-1.1B may lack the parameter budget to absorb instruction tuning without distributional drift on these benchmarks, though a single dataset and two scales cannot rule out other explanations.

The same procedure improves substantially on the objective it optimizes. On the held-out Alpaca slice the base model scores 1.9352 (perplexity 6.9254) and every fine-tuned checkpoint scores between 1.3349 and 1.3608 (perplexity 3.80 to 3.90). The regression is a divergence between the training objective and these benchmarks rather than a training failure, and it is why quality is reported here as held-out instruction-following loss. The held-out slice is drawn from the same distribution as the training data, so it measures instruction following on unseen examples and is not evidence of generalization to unrelated tasks.

The regression is independent of aggregation method. Cross-method spread at TinyLlama scale is 0.006 on ARC-Easy, 0.010 on BoolQ, and 0.000 on HellaSwag, so the zero-shot benchmarks do not separate protocols that the held-out metric separates at $p < 0.05$. These accuracies are single-seed (seed 42, MPS corpus), unlike the three-seed CUDA results in Table II, so the spreads carry no variance estimate and indicate insensitivity rather than establishing equivalence.

The BoolQ result admits a specific account. The base model's accuracy of 0.626 closely matches BoolQ's natural yes-class rate of approximately 0.62, suggesting the base exploits a yes-biased prior. Instruction tuning attenuates that prior, which is why BoolQ moves furthest of the three.

VI. Limitations

The experiments use two instruction-following datasets (Alpaca-3k [21] and Dolly-15k [22]) and a single task type. Generalization to classification tasks, longer training runs, or generation-quality metrics is not characterized. Quality is measured as held-out loss on examples drawn from the same distribution as the training data, which captures instruction following on unseen examples rather than transfer to unrelated tasks.

The two published baselines, FFA-LoRA and FedIT, were evaluated on Alpaca only, so Table III compares three protocols where Table II compares five. The cross-dataset result therefore speaks to the cost of the adaptive transition rather than to the stability of the whole frontier across datasets.

The four zero-shot benchmarks do not discriminate between aggregation protocols at TinyLlama scale, where cross-method spread is at most 1.0 percentage point and every fine-tuned checkpoint scores below the base model (Section V-D). Conclusions about relative protocol quality therefore rest on held-out instruction-following loss alone.

The simulation assumes full client participation in every round. Real federated deployments typically involve partial participation, where only a fraction of clients respond per round. The effect of partial participation on transition-round timing and on ReverseAdaptive's plateau detection is not studied here.

Non-IID partitions are constructed by Dirichlet skew over instruction-length buckets rather than over semantic labels, since neither dataset carries class labels. This induces heterogeneity in task form. Heterogeneity in task semantics, which is what semantic-class partitioning models in classification settings, is untested.

The study covers only Llama-style decoder-only architectures with LoRA targeting `q_proj` and `v_proj`. Other architectures, encoder-decoder models, larger target sets, or heterogeneous rank configurations may produce different B-only fractions and different savings profiles. The reported percentages are specific to this target set and to the grouped-query attention configurations of the two models tested.

At the transition round the server freezes A from the first client's uploaded state. Freezing the aggregated global A that the server already holds from the preceding round would avoid the seeding upload entirely and is untested, so the 55.0 MB transition cost reported here is a property of this implementation rather than of phase-switching protocols in general (Section III-D).

Runs were produced across multiple code revisions with uncommitted local modifications. The primary CUDA corpus spans two revisions, and the FLoRA and FedIT rows of Table II sit on either side of that boundary while remaining statistically indistinguishable, which indicates that the combined algorithmic and revision difference across that boundary is roughly 0.0006 in final loss. The LLaMA-3.2-3B results are not single-revision: the three seeds were produced at three

revisions, so the variability reported there combines seed and environment variation.

LLaMA-3.2-3B experiments use three random seeds with downstream evaluation per checkpoint. Seed-to-seed evaluation noise within a single checkpoint is not separately characterized. At TinyLlama-1.1B, downstream evaluation is single-seed per checkpoint while training uses three seeds.

No real-world network conditions are simulated. Byte counts are measured from the tensors actually transmitted, but network latency, packet loss, and bandwidth constraints are not modeled, so wall-clock communication time is not predicted.

No differential privacy analysis is included. FFA-LoRA [5] was originally motivated in part by DP-friendliness; the transition-round dynamics of the bidirectional B-only protocol may affect DP accounting in non-obvious ways that warrant separate study.

Future work should address partial participation, non-Llama architectures, semantic heterogeneity, and real network conditions to validate deployment claims more broadly. Composing the bidirectional B-only protocol with gradient compression techniques [12], [13] for savings beyond the B-only floor is a natural extension. Per-layer adaptive phase-switching, where different transformer layers switch at different rounds based on their individual loss-plateau signals, is another direction.

VII. Conclusion

This paper's central contribution is a shift from theoretical parameter-count savings to measured byte-level reporting for federated LoRA. Tracking per-round upload and download megabytes, including at the asymmetric transition boundary, shows that the transition costs a fixed 55.0 MB on TinyLlama-1.1B regardless of when it occurs, a quantity parameter-count accounting cannot express.

Those measurements place five published protocols on a single communication-quality frontier with a knee. ReverseAdaptive sits at that knee and locates it without a phase boundary being specified in advance; the marginal cost of moving past it is roughly five times higher per point saved.

The same threshold transfers across model scales without retuning, firing at round 6 on TinyLlama-1.1B and at a mean of round 8 on LLaMA-3.2-3B, and the quality cost of the transition is stable across the two datasets tested.

As open-weight LLMs continue to grow in capability and federated deployments expand into privacy-sensitive domains, the communication bottleneck identified by Kairouz et al. [1] will only intensify. Reporting what protocols actually transmit, rather than what a parameter count predicts, is a prerequisite for comparing them.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

All code, configuration files, and analysis scripts are publicly available at https://github.com/jerryadamsfranklin/fedlora-protocols. Adapter checkpoints are not released. The Alpaca and Dolly-15k datasets are publicly available from their respective sources.

The author used Claude (Anthropic) for grammar and language editing only. All technical content, analysis, and interpretations are the author's own.

References

[1] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[3] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, G. Wang, and Y. Chen, "Towards building the FederatedGPT: Federated instruction tuning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 6915–6919, arXiv:2305.05644.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[5] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving LoRA in privacy-preserving federated learning," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2403.12313.

[6] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2409.05976.

[7] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous language tasks and client resources (FlexLoRA)," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.11505.

[8] G. Yan, L. Xie, Q. Shen, Y. Fang, and Z. Wu, "FedSRD: Sparsify-reconstruct-decompose for communication-efficient federated large language models fine-tuning," in *Proceedings of the ACM Web Conference 2026 (WWW '26)*, 2026, arXiv:2510.04601.

[9] H. Ramesh and J. Dass, "FLoRIST: Singular value thresholding for efficient and accurate federated fine-tuning of large language models," in *Proceedings of the Ninth Conference on Machine Learning and Systems (MLSys)*, 2026, arXiv:2506.09199.

[10] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning (ICML)*, 2019.

[11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-efficient SGD via gradient quantization and encoding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[13] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep gradient compression: Reducing the communication bandwidth for distributed training," in *International Conference on Learning Representations (ICLR)*, 2018.

[14] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2020.

[15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2020.

[16] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization (FedNova)," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] S. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," in *International Conference on Learning Representations (ICLR)*, 2021.

[18] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[19] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," *arXiv preprint*, 2024, arXiv:2401.02385.
[20] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle *et al.*, "The Llama 3 herd of models," *arXiv preprint*, 2024, arXiv:2407.21783.
[21] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Alpaca: A strong, replicable instruction-following model," Stanford CRFM Blog, 2023.
[22] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin, "Free dolly: Introducing the world's first truly open instruction-tuned LLM," https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm, 2023.
[23] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021.
[24] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," in *arXiv:1803.05457*, 2018.
[25] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the surprising difficulty of natural yes/no questions," in *Proceedings of NAACL-HLT*, 2019.
[26] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a machine really finish your sentence?" in *Proceedings of the Association for Computational Linguistics (ACL)*, 2019.